

ZIPLO / GOKIRANA

Hyperlocal Quick-Commerce & Kirana Digitization Platform Documentation

Document Version: 1.0.0	Date: August 2026
Project Architecture: FastAPI + React (Vite) + SQLite/AWS S3	Target Market: Tier-2/3 Indian Cities (Vadodara)
Author/Team: Webs Sonic / Snehil Pallav Sinha	Status: Production Ready & Functional

Executive Overview & Business Mission

Ziplo (GoKirana) is an end-to-end quick-commerce and local retail digitization ecosystem engineered to empower neighborhood Kirana stores, pharmacies, and specialty retailers in Tier-2/Tier-3 cities. It enables local merchants to offer hyper-fast 10-to-30 minute deliveries to customers within a hyper-local radius (0-5 km), competing directly with dark-store instant delivery giants while preserving local merchant livelihoods.

The platform bridges local buyers, store owners, and delivery partners through real-time order routing, live GPS tracking, automated OTP-based delivery verification, localized SMS notifications, dynamic inventory management, and an operational analytics dashboard.

Document Structure

- **Section 1: Architecture & Technology Dependencies** — Stack details, python packages, frontend libraries.
- **Section 2: Comprehensive Core Features** — Multi-role capabilities for Customers, Store Owners, Delivery Agents & Admins.
- **Section 3: End-to-End User & Operational Flows** — Complete lifecycles for registration, ordering, live tracking, and status transitions.
- **Section 4: Data Models & Database Schema** — Relational tables, fields, types, foreign keys, and indexes.
- **Section 5: API Endpoint Directory & Protocols** — Detailed REST API endpoints, WebSockets, S3 upload logic, and SMS integration.
- **Section 6: Deployment & Operational Runbook** — Environment configurations, startup scripts, and background services.

1. Architecture & Technology Dependencies

System Architecture Overview

Ziplo follows a modern decoupled client-server architecture. The backend is built with high-performance **FastAPI (Python)** exposing asynchronous RESTful APIs and full-duplex **WebSockets** for real-time order status and GPS updates. Data persistence is managed via **SQLAlchemy ORM** connected to an optimized **SQLite database (gokirana.db)**. The frontend is a responsive Single-Page Application (SPA) powered by **React 19**, **Vite 8**, and **Lucide React** icons.

Backend Dependencies (Python / FastAPI Stack)

Package Name	Version / Range	Functional Role / Usage
FastAPI	^0.110.0	Core asynchronous web framework providing REST endpoints, WebSocket routing, dependency injection, and OpenAPI documentation.
Uvicorn	^0.28.0	Lightning-fast ASGI server implementation for handling async HTTP and WebSocket connections.
SQLAlchemy	^2.0.0	Object-Relational Mapping (ORM) library for schema declaration, relationship mapping, and SQL query generation.
Pydantic	^2.6.0	Data validation and serialization library defining strict input request schemas and output API models.
Boto3	^1.34.0	AWS SDK for Python, integrated for secure image asset uploads to AWS S3 buckets.
python-dotenv	^1.0.0	Loads environment secrets (.env) into OS environment variables (S3 credentials, Fast2SMS API keys).
requests	^2.31.0	HTTP client library used by the SMS service layer to communicate with Fast2SMS REST gateway.
ReportLab	^4.4.0	PDF generation library used for building programmatic project documentation and invoices.

Frontend Dependencies (React / Node Stack)

Dependency	Version	Functional Description
React / React-DOM	^19.2.6	UI Component rendering library utilizing virtual DOM and modern functional hooks (useState, useEffect, useMemo).
Vite	^8.0.12	Next-generation frontend build tool and dev server featuring instant HMR (Hot Module Replacement).
Lucide React	^1.17.0	Modern, clean SVG icon library providing intuitive UI icons for cart, navigation, GPS, shops, and metrics.
ESLint / Plugins	^10.3.0	Static code analysis and linting tools ensuring code quality, style consistency, and hook rules safety.

2. Comprehensive Core Features & Capabilities

1. Customer Module Capabilities

- **Hyperlocal Shop Discovery:** Calculates real-time distance between customer GPS coordinates and local Kirana stores using Haversine distance, prioritizing shops within a 0-5 km radius.
- **Category-Based Product Browsing:** Instant filtering across multiple local inventory categories including Grocery, Pharmacy, Baby Care, Pet Care, and Stationery.
- **Dynamic Cart & Order Checkout:** Add items, adjust quantities with live price calculation, auto-apply delivery fee calculation based on distance, and select Cash on Delivery (COD) or UPI.
- **Live Order Status Tracking:** Visual progress timeline showing order state transitions (Pending → Accepted → Packing → Out for Delivery → Delivered).
- **Real-Time GPS Delivery Tracking:** Interactive simulation/live broadcast of delivery partner coordinates moving along route to customer delivery location.
- **Delivery OTP Verification:** Displays a unique 4-digit security PIN for secure handover to the delivery agent.
- **Customer Profile & Order History:** Seamless phone number lookup with cached customer address and location preferences.

2. Merchant / Shop Owner Module Capabilities

- **Real-Time Order Dashboard:** Live WebSocket feeds notify store managers immediately when a new customer order is placed.
- **One-Click Order Workflow Management:** Shop owners can Accept, Mark as Packed, and Dispatch orders for delivery.
- **Digital Catalog & Inventory Management:** Add new products with image upload support (AWS S3 integration or pre-hosted URL fallbacks), edit MRP/offered prices, update stocks, or delete items.
- **Merchant Analytics Summary:** View daily total orders, revenue generated, and active inventory status.

3. Delivery Agent & Operations Module Capabilities

- **Order Allocation Portal:** View dispatched orders ready for pickup with shop pickup address and customer drop address.
- **GPS Location Broadcaster:** Live broadcast of delivery agent latitude/longitude coordinates via WebSocket/API updates.
- **OTP Handover Verification:** Enforces input of customer's 4-digit OTP prior to marking an order as 'Delivered', preventing false or lost deliveries.

4. Admin Oversight & Master Analytics Dashboard

- **Platform Health Metrics:** Total platform revenue, total order volume, active shops count, and completed delivery ratios.

- **Global Shop Directory:** View all onboarded stores, phone contacts, shop addresses, and exact lat/long coordinates.
- **Global Order Log & Audit Trail:** Monitor all system transactions across stores in real time.

3. End-to-End Operational & System Flows

Flow A: Customer Order Placement & Checkout Lifecycle

1. **Location Detection:** Customer opens app; browser requests HTML5 Geolocation. App resolves lat/long (default: Alkapuri, Vadodara 22.3072, 73.1678).
2. **Store & Item Selection:** App queries GET /api/shops. Distances are computed. Customer selects store, views items via GET /api/shops/{id}/products, and adds items to cart.
3. **Checkout Payload:** Customer provides Name, Phone Number, and Delivery Address. Cart items are formatted as JSON string.
4. **Backend Execution:** Frontend fires POST /api/orders.
 - a. System looks up or creates Customer record.
 - b. System generates random 4-digit delivery_otp (e.g. 4821).
 - c. Order created with status Pending.
 - d. SMS sent to customer with OTP & confirmation link via Fast2SMS.
 - e. WebSocket event broadcasted to shop channel /ws/shop/{shop_id}.

Flow B: Merchant Order Processing & Dispatch Lifecycle

1. **Alert Received:** Shop dashboard receives real-time WebSocket packet or polls GET /api/orders/shop/{shop_id}.
2. **Order Acceptance:** Merchant clicks 'Accept Order'. API call PUT /api/orders/{id}/status updates status to Accepted.
3. **Packing Items:** Store staff gathers items and updates status to Packing.
4. **Dispatch:** Store marks order as Out for Delivery. Order becomes visible in the Delivery Agent Portal.

Flow C: Live Delivery, GPS Tracking & OTP Verification Lifecycle

1. **Agent Pick Up:** Delivery partner accepts assignment and starts navigating to customer.
2. **Live GPS Broadcast:** Delivery partner app periodically posts location updates via PUT /api/orders/{id}/location with lat/long.
3. **Customer Map Stream:** Customer tracking UI polls customer orders GET /api/orders/customer/{phone} or listens to WebSocket, rendering moving marker on map.
4. **Handover & Completion:** Delivery partner arrives at doorstep and asks customer for 4-digit OTP.
 - a. Partner enters OTP into Delivery App UI.
 - b. System validates OTP against database record.
 - c. If matched, status updates to Delivered and delivery timestamp is recorded.

4. Data Models & Database Schema

Relational Database Schema Diagram (SQLite)

Table Name	Primary Key & Fields	Relationships & Foreign Keys
shops	• id (Integer, PK) • name (String) • phone (String, Unique) • password (String) • address (String) • coordinates (String) • image_url (String)	• Has many products (1:N) • Has many orders (1:N)
products	• id (Integer, PK) • shop_id (Integer, FK) • name (String) • description (String) • mrp (Float) • offered_price (Float) • stock (Integer) • category (String) • image_url (String)	• Belongs to shop (FK: shops.id)
customers	• id (Integer, PK) • name (String) • phone (String, Unique, Index) • address (String) • coordinates (String)	• Has many orders (1:N)
orders	• id (Integer, PK) • customer_name/phone/address • shop_id (Integer, FK) • items (Text/JSON String) • total_amount (Float) • status (String: Pending, Accepted, etc.) • delivery_otp (String) • live_location (String) • created_at (DateTime)	• Belongs to shop (FK: shops.id) • Belongs to customer (via phone lookup)

5. Complete API Endpoint Directory

Method	Endpoint Path	Description & Parameters
GET	/api/shops	Returns list of all active Kirana stores with addresses, phone numbers, and coordinates.
GET	/api/shops/{id}/products	Retrieves inventory catalog for a specific shop filtered by shop ID.
POST	/api/products	Creates a new product item for a shop. Supports fields: shop_id, name, price, stock, category.
PUT	/api/products/{id}	Updates product details (MRP, offered price, stock availability, category, image URL).
DELETE	/api/products/{id}	Deletes a product item from the shop catalog.
POST	/api/orders	Places a new order. Accepts customer details, shop_id, cart items JSON. Generates OTP & triggers SMS.
GET	/api/orders/customer/{phone}	Fetches active and historic orders for a specific customer phone number.
GET	/api/orders/shop/{shop_id}	Retrieves all incoming and processed orders for a specific merchant store.
PUT	/api/orders/{id}/status	Updates order status enum: Pending → Accepted → Packing → Out for Delivery → Delivered.
PUT	/api/orders/{id}/location	Updates live GPS coordinates (lat,long string) of the delivery partner for live tracking.
POST	/api/upload	Uploads product image file to AWS S3 bucket and returns public S3 object URL.
POST	/api/seed	Populates database with sample Kirana shops (Alkapuri, Gotri, Manjalpur) and grocery catalog items.
GET	/api/admin/metrics	Computes master admin platform statistics (total sales revenue, order counts, active shops).
WS	/ws/shop/{shop_id}	Full-duplex WebSocket connection for real-time order alerts to store manager UI.

6. Operational Setup & Launch Runbook

Execution Script (run_project.bat)

```
@echo off
echo Starting Ziplo Project Servers...

:: Start FastAPI Backend on port 8000
start cmd /k "uvicorn backend.main:app --reload --port 8000"

:: Start React Vite Frontend on port 5173
cd frontend
npm run dev
```

Environment Variables Configuration (`.env`)

```
AWS_ACCESS_KEY_ID=your_aws_access_key
AWS_SECRET_ACCESS_KEY=your_aws_secret_key
AWS_REGION=ap-south-1
S3_BUCKET_NAME=ziplo-product-images
FAST2SMS_API_KEY=your_fast2sms_api_key
```